



Stargazer Axiom Guidelines

Welcome! This document outlines the process and structure for creating issues for the Stargazer Axiom project, including code base preparation, issue creation, solution development, test writing, and setting up Docker files and execution scripts.

Table of Contents (TOC)

Project Overview	2
Task Overview	3
Step 0: Onboarding & Installations (return to TOC)	3
Project Onboarding (return to TOC)	3
Environment Setup (return to TOC)	4
Project Structure (return to TOC)	4
Component Descriptions (return to TOC)	5
Step 1: Setup and Analysis (return to TOC)	6
1.1 Environment Setup (return to TOC)	6
Step 2: Issue Creation and Solution (return to TOC)	7
2.1 Write Issue Statement (return to TOC)	7
2.2 Rate the difficulty of your issue (return to TOC)	9
2.3 Create the Issue (return to TOC)	9
2.4 Test the Issue Message (return to TOC)	10
2.5 Generate the Solution and Gold Patch (return to TOC)	10
Step 3: Testing (return to TOC)	11
Expected Test Behavior (return to TOC)	12

3.1 Write Fail-to-Pass (F2P) Tests (return to TOC)	12
3.2 Write Pass-to-Pass (P2P) Tests (return to TOC)	13
3.3 Generate the Test Patch (return to TOC)	13
Step 4: Docker and Scripts (return to TOC)	14
4.1 Create Run Script (return to TOC)	14
4.2 Create Parse Script (return to TOC)	15
4.3 Create Base Dockerfile (return to TOC)	16
4.4 Create Instance Dockerfile (return to TOC)	17
Step 5: Running the Validation Script (return to TOC)	17
Step 6: Add tags (return to TOC)	19
Step 7: Rubric Creation (return to TOC)	20
Guidelines (return to TOC)	20
Core Principles of Rubric Criteria (return to TOC)	21
Quality Dimensions (return to TOC)	21
Key Requirements (return to TOC)	22
Generate your rubric.json (return to TOC)	22
Step 8: Evaluation Script (return to TOC)	23
How to run it (return to TOC)	23

Changelog

 Date	 Updates
June 10	→ NEW: Step 0 - Project Onboarding
May 28	→ model_trace.patch is now required for model failure trace

Project Overview

The purpose of this project is to create a **database of challenging issue requests** for a repository. This database will be used to benchmark and improve the ability of coding agents to implement changes in a codebase in response to user prompts.

To achieve this, you will build a standard project setup around an assigned repository. You will then create a challenging prompt requesting changes to the codebase. You will check that the model fails to respond adequately to your prompt, write your own solution to the issue, create unit tests to verify your solution, and compose a rubric that can be used to grade how successful the agent was in responding to your issue. When you are finished, you will prepare the required patches, scripts, and Docker files so the whole workflow can be reproduced and validated later.

Task Overview

Description	
0 Onboarding & Installations	Before tasking for the first time, you must complete the project onboarding and reach out to the project team after completing your quality assessment. Then you will set up Cursor and Docker. You only need to do this once!
1 Setup and Analysis	First, you will clone the assigned repo and set up your environment. You will also explore the codebase so that you understand its structure and functionality.
2 Issue Creation and Solution	You will write a prompt that asks the agent to make changes to the codebase based on an assigned issue (e.g., bug fixes or new features). You will make sure that your prompt is challenging enough that at least one model fails to solve it. Then, you will write your own Golden Solution to your prompt.
3 Testing	You will create unit tests that can check whether a given solution adds all of the requested functionality without breaking any previously existing functionality.
4 Docker and Scripts	You will prepare Dockerfiles and scripts that ensure that your work on the task is reproducible.
5 Validation Script	You will run a validation script that checks that your solution passes all of the unit tests that you wrote.
6 Add Tags	You will add relevant tags to your task to categorize it.
7 Rubric Creation	You will write a rubric that consists of a series of criteria that can grade how good or bad a response to your prompt is.
8 Evaluation Script	Before submission, you will run an evaluation script on your task. The suite checks the quality and alignment of your prompt, tests, rubrics, gold patch, Dockerfiles, and granularity.

Step 0: Onboarding & Installations [\(return to TOC\)](#)

Project Onboarding [\(return to TOC\)](#)

Before your first task, you will be guided through our **onboarding** process, where you will learn the basics of what you need to know to succeed on the project.

As part of the onboarding process, you will submit a **quality assessment**.

Please reach out to the project team in the [Zoom War Room](#) after completing the quality assessment. This step is **mandatory** for you to begin tasking. The project team will need to perform a manual review to ensure that you passed the quality assessment and to provide access to Cursor.

Environment Setup [\(return to TOC\)](#)

Before getting started with the project, both Cursor and Docker must be properly set up and running. Here is a quick guide to make sure your environment is properly set up.

Cursor

- Follow this [link](#) and **download the 2.6 cursor version**. If you already have Cursor installed, uninstall it and install this specific version, without this version, **the models won't work**.
- Even if you already have access to the **Cursor Team Plan**, reach out to the [Zoom War Room](#) and ask any of the QMs for a new plan, as it must be renewed for this project.
- Go to the Key Manager: <https://cursor-intelligence-service.outlier.ai/key-manager>
- Click on View API Key
- Open Cursor Settings → Models
- Disable all models, and add:
 - claude-qwen3-5-27b-scale
 - claude-4.6-sonnet-scale
- Enable OpenAI API Key; paste in your API Key and Override Base URL with <https://cursor-intelligence-api.outlier.ai/api/v1>
- Click "Verify".
- Test these models in Cursor now.

Docker

- Make sure Docker is installed and up to date. If you don't have it yet, follow [this guide](#).
- Ensure Docker has enough RAM and disk space to run containers smoothly.
- Clean up unused images and containers regularly to free up space and avoid build errors.

Project Structure [\(return to TOC\)](#)

It is important to organize the project with a **clear and consistent structure** from the start. This will make it easier to manage the repository, track the issue and fix, and ensure that everything runs correctly.

(MANDATORY) The file structure for this project must look like this:

None

```
project/
├─ App/
├─ dockerfiles/
│   ├─ base.Dockerfile
│   └─ instance.Dockerfile
├─ scripts/
│   ├─ run_script.sh
│   └─ parse_results.sh
├─ patches/
│   ├─ gold_patch.patch
│   ├─ test_patch.patch
│   └─ basetoinstance.patch
└─ test_results.json
```

Component Descriptions [\(return to TOC\)](#)

1. App

- The project assumes a single repository, cloned directly from GitHub. You can use this folder to clone the repo in your local environment to design issues, create failing tests, and generate solution patches.

2. Dockerfiles

- **Base Dockerfile (base.Dockerfile):** This Dockerfile sets up the environment by creating `/app`, cloning the repository into it, and installing any required system or project dependencies.
- **Instance Dockerfile (instance.Dockerfile):** This Dockerfile extends the base image and applies `patches/basetoinstance.patch` to `/app`. The build logic must handle the patch file, whether it is populated (this will be the case in Bug Injection tasks) or empty (this will be the case in all other tasks). In the case of Bug injection, the patch serves to transition the codebase into the designated problem state for evaluation.

3. Scripts

- The `scripts/` directory contains executable shell scripts (`run_script.sh`, `parse_results.sh`) that are mounted into the container under `/app/scripts`. These scripts are responsible for applying additional patches, running tests, and parsing results.

4. Patches

- The `patches/` directory defines all transformations applied to the repository at `/app`.
 - `basetoinstance.patch` modifies the cloned repository to represent the problem scenario.
 - `test_patch.patch` introduces tests that should fail before your patch is applied.
 - `gold_patch.patch` introduces your solution to the issue, which should pass all the unit tests.

5. JSON-Formatted Tests Results

- The `test_results.json` output from `parse_results.sh`

Step 1: Setup and Analysis [\(return to TOC\)](#)

1.1 Environment Setup [\(return to TOC\)](#)

Before reviewing the assigned codebase, make sure your environment is set up, Cursor is configured, and Docker is running. Review [Environment Setup](#) section in Step 0 for detailed setup instructions.

Create the following folders in your working directory:

- App
- dockerfiles
- scripts
- patches

1.2 Clone Repo & Analyze Codebase [\(return to TOC\)](#)

First, open a new terminal and clone the given task repo using the provided **repository clone command**. Rename the cloned folder as **App/**. After that, explore the codebase you cloned:

- Review the code structure and architecture
- Understand dependencies and build process
- Understand the issue type you were assigned for this task (Review [Issue Types Examples](#) section in the Appendix for specific examples):
 - **New feature request:** Adding a new functionality not currently in the given code.
 - **Bug injection + resolution:** Introducing a breaking change to working code
 - **Migration:** Adapting all or part of the existing code to a new library/pattern/version without losing its original functionality.
 - **Performance Optimization:** Improve speed, efficiency, or resource usage with no changes to the original functionality.
 - **Maintenance or testing enhancement:** Updating dependencies, tests, or tooling

Step 2: Issue Creation and Solution [\(return to TOC\)](#)

2.1 Write Issue Statement [\(return to TOC\)](#)

After reviewing the codebase, you will create a prompt based on the issue type that you were assigned. The prompt should make a request to the agent based on your issue type. It should also be complex enough that it challenges the models (**Sonnet 4.6** and **Qwen 3.5 27b**) so that one of them fails. You can see examples of the different issue types in [Issue Types Examples](#) section in the Appendix.

Prompt Guidelines:

- Specific enough that it is possible for the model to succeed
- Not so detailed that it gives away the solution to the model

- Should challenge models like **Sonnet 4.6** and **Qwen 3.5 27b**.
- Simple and clear, then add complexity if the model succeeds

Prompt and Rubric Alignment:

At the end of the task, you will write a rubric that can be used to grade at a granular level how well the model did in solving your issue. The prompt and the rubric need to be aligned on their level of specificity. This means:

- Prompts must explicitly mention filenames, paths, functions, or modules if the rubric criteria reference them. If the rubric lists specific paths or APIs, the prompt must also contain those exact references.
- It is acceptable to make the prompt unrealistically specific if that is necessary to align with the level of granularity in the rubric and guarantee that the rubric criteria unambiguously evaluate the solution

How to build a good prompt:

The prompt is the foundation of the task. Its quality determines the quality of every component built on top of it (test patch, golden patch, and rubrics). Treat this step carefully.

The goal of the prompt is to realistically simulate an issue a developer would encounter while modifying or improving a repository. Write from that perspective: a real request, grounded in the repo, phrased the way a developer would actually phrase it.

Follow these principles:

- **Write it like a real GitHub issue.** The prompt should read like something a developer would actually open in a repo focused on **one** problem or request. Try **not to** stack multiple unrelated changes, pile on extra features, or ask for several different things in the same prompt. One issue, one ask. If you find yourself writing "and also," stop and split the idea or cut it.
- **Be clear and concise.** State the problem directly. Difficulty should come from the realism of the problem, not from artificial complexity or volume.
- **Eliminate major ambiguity.** Before finalizing, reread the prompt and ask whether two competent developers would interpret the requirements the same way. If not, tighten the wording.
- **Use explicit requirements.** Every requirement must be concrete enough that it can be verified by a test and satisfied by the gold path solution. Avoid vague language ("make it better," "improve performance") unless the success criteria are clearly implied by context.

- **Think ahead.** Before writing, plan how the gold solution and tests will look. If you cannot picture them, the prompt is not ready.

2.2 Rate the difficulty of your issue [\(return to TOC\)](#)

Rate your issue by counting how many of the following complexity **signals** apply:

1. **Spread:** The fix changes **3 or more files**.
2. **Cross-module interaction:** The fix affects **more than one part of the project**.
3. **Architectural awareness:** To fix it, you need to understand **how the project is organized**.
4. **Non-trivial logic:** The fix needs **decision-making in the code**, not just renaming, moving, or deleting something.
5. **Implicit edge cases:** The solution should also handle **special cases or possible errors**.
6. **Regression risk:** A bad fix could **accidentally break something else**.

Difficulty	Signals that apply
Easy	1 - 2
Medium	3 - 4
Hard	5+

2.3 Create the Issue [\(return to TOC\)](#)

Location: **App/**

- For all issue types except Bug Injection, the files inside **app/** must **remain identical** to the original repo.
- For **Bug Injection**, introduce the bug directly inside the **app/** folder so that this modified version represents the faulty instance state that the solution will fix.

2.3.1 Generate Base-to-Instance Patch

You must generate an additional patch called **basetoinstance.patch**. If the issue type is Bug Injection, after introducing the bug, you need to capture the exact diff between the working **base** state and the broken **instance** state

If it's not bug injection, please upload an empty basetoinstance.patch file to the task
(Please note that the Outlier platform does not allow literally empty files. Adding an empty line or a space is acceptable)

2.4 Test the Issue Message [\(return to TOC\)](#)

Make sure the issue message is challenging enough to cause at least **one model to fail**. To do so, open only the **App/** folder in Cursor and have both models available: **Sonnet 4.6** and **Qwen 3.5 27b**, if you don't have them available go to the [Environment Setup section in the appendix](#) to see a full guide on how to set them up.

- If it is a Bug injection task, make sure the **app/** folder contains the bug
- Paste the issue message and select one model (**Sonnet 4.6** or **Qwen 3.5 27b**). Then check whether it fails to create a valid solution.
- Repeat the same with the other model.
- If both models solve the issue correctly, rewrite or adjust the issue message to increase its complexity or change the issue topic until one of them fails.
- Ensure that the reported failure is directly tied to a specific requirement described in the issue message. The model should only fail when it does not correctly implement an expected behavior explicitly requested in the prompt, not because of assumptions, inferred expectations, or invented edge cases that are not part of the original issue description. If you want to know about valid failures refer to the [Sample Invalid Model Failures](#) section.
- When at least one model fails, and the issue message meets all requirements, select "Yes" and continue with the task.
- Extract the model failure trace from Cursor as documentation. (If both models fail, capturing one of them is enough). If you need a guide on how to generate that trace, go to the [How to generate the model trace](#) section.
- Generate a model_trace.patch containing the exact changes made by the model during its trace. If you need a guide on how to generate that file, go to the [How to generate model_trace.patch](#) section.



IMPORTANT: Ensure that only the **App/** folder is opened in Cursor so the models do not have access to the patches with the solution (you can open a new Cursor window with just the **App/** content).

2.5 Generate the Solution and Gold Patch [\(return to TOC\)](#)

Once **Sonnet 4.6** or **Qwen 3.5 27b** fails, you will create your own Golden Solution that represents a complete, flawless solution to your prompt issue.

Location: `patches/gold_patch.patch`

Implement your solution on the issue branch, then create a patch:

Shell

```
# Make your changes to implement the solution
# Edit the necessary files...

# Create the patch and save it to the patches folder
# Option A - Specific files:
git diff <file1> <file2> ... > patches/gold_patch.patch

# Option B - All changes (ensure no unrelated edits):
git diff > patches/gold_patch.patch

# Undo the changes to keep branch in problem state
# Option A - Specific files:
git checkout -- <path/to/file>

# Option B - All files:
git checkout -- .
```

If you need a more detailed step-by-step on how to generate a .patch file, you can follow the [How to generate .patch files guide](#).



Important: Make sure the patch can be cleanly applied using `git apply --ignore-whitespace`

Step 3: Testing [\(return to TOC\)](#)

In this step, you must write tests to validate if ANY implemented solution (including the Golden solution you wrote yourself) solves your issue statement. For this, you will write Fail-to-Pass (F2P) and Pass-to-Pass (P2P) behavioral tests, meaning no check of structure / integration is required. The goal of the testing is to make sure that the code produces the correct behaviors; you should NOT test any specific implementation with your tests unless it was specifically requested in your prompt. Any solution that creates the correct behaviors without breaking any existing functionality should pass all tests. Additionally, remember that your tests need to be **self-contained**; they cannot depend on other things to work as expected.

Expected Test Behavior [\(return to TOC\)](#)

Test Type	Before Fix	After Fix	Purpose
F2P	✗ Fail	✓ Pass	Verifies new behavior
P2P	✓ Pass	✓ Pass	Confirms existing features still work

3.1 Write Fail-to-Pass (F2P) Tests [\(return to TOC\)](#)

Location: Use `**/*.f2p.test.{js,ts,php,etc}` filenames so the parser can detect them. Use these **exact** suffixes: parse scripts rely on them. Use the [parse_results.sh template](#) in the Templates document.

Create tests that validate your solution:

Characteristics:

- ✗ FAIL on the problem state (before applying the gold patch)
- ✓ PASS after applying the gold patch
- Test the specific changes/functionality added by your solution

Test examples by issue type:

- Bug Injection:** Tests that verify the bug is fixed (e.g., layout switching updates state correctly)
- New Feature:** Tests that verify the new feature exists and works (e.g., export button generates a valid CSV)
- Migration:** Tests that verify the new library/pattern exists and is used correctly (e.g., new auth API is called)
- Performance:** Benchmark tests showing improvement and/or tests that confirm that updated code meets standards (e.g., rendering completes in <100ms)

- **Maintenance:** Tests that verify new tooling works or confirm any updated dependencies, documentation, refactoring, etc (e.g., linting rules are enforced)
- **Testing Enhancement:** Tests that verify test files exist.



Important: For Testing Enhancement tasks, F2P tests should verify test files exist with correct structure, naming conventions, and basic executability.

3.2 Write Pass-to-Pass (P2P) Tests [\(return to TOC\)](#)

Location: Use `**/*.p2p.test.{js,ts,php,etc}` filenames so the parser can detect them. Use these exact suffixes; parse scripts rely on them. Use the [parse_results.sh template](#) in the Templates document.

Create tests for existing functionality:

Characteristics:

- ☒ PASS on the problem state (before applying the gold patch)
- ☒ PASS after applying the gold patch
- Test functionality around/related to the changes
- Ensure no regression or side effects



Important: For Testing Enhancement tasks, P2P tests should validate the testing infrastructure hasn't regressed (e.g., test framework setup, test utilities, CI/CD pipeline), not the application functionality being tested by the new tests.

3.3 Generate the Test Patch [\(return to TOC\)](#)

Location: `patches/test_patch.patch`

After writing both test suites, create a patch containing them:

Shell

```
# 1. Create test files
# Write your F2P and P2P tests...

# 2. Create patch directly from working directory
git diff > patches/test_patch.patch
```




Important:

- ✓ Make sure the patch can be cleanly applied using `git patch apply --ignore-whitespace`
- ✓ The **test_patch.patch** must include *only test files*. If you need any configuration files for testing, you **MUST** include them here too (This does not include the testing dependencies, and they have to be on the base.Dockerfile
E.g. Mock data to run tests, jest.config, etc

test_patch.patch must NOT contain changes to application code, existing tests, dependencies, or config files, such as:

- `package.json`
 - `package-lock.json`
 - Source files outside the test directory
 - Files unrelated to testing
- ✓ The **test_patch.patch** can run `npm install` if your task requires modifications to the `package.json`.

Step 4: Docker and Scripts [\(return to TOC\)](#)

In this step, you will create scripts and a Dockerfile that ensure that your work is reproducible.

4.1 Create Run Script [\(return to TOC\)](#)

- Location: `scripts/run_script.sh`
- Template: Use the [run_script.sh template](#) in the Templates document as a starting point and modify it as needed.
- The script should:
 1. **Apply the test patch:** The script should **apply the test patch before running any tests**.
 2. **Run tests on broken state (Phase 1):** It should **run the tests on the broken code first** and **show the full output**.
 3. **Apply the gold patch:** It should then **apply the gold patch after Phase 1**.

4. **Run tests on fixed state (Phase 2):** It should **run the tests again on the fixed code** and **show the full output**.
5. **Run cleanly:** The script should **complete successfully using the expected flow**, without **patch application issues or script-level failures**.
6. For **Migration** tasks that require installing new dependencies or modifications to the package.json, please check [Installing dependencies in test_patch.patch](#) section in the Appendix for a how-to-do-it guideline.

IMPORTANT: All the repository dependencies and the necessary assets (env variables, etc) should be installed/configured in the `base.Dockerfile`.



This also includes **extra test-only dependencies** that are needed to run the tests, even if they are **not included in the original repository** and are **not added by the gold patch**. For example, if the tests need `playwright`, it and anything else required for it to run **must be set up in the `base.Dockerfile`**.

The `run_script.sh` should be focused **ONLY** on applying the patches and running the tests for each phase.

It must not install ANY dependencies or run `npm install` (or similar).

4.2 Create Parse Script [\(return to TOC\)](#)

- Location: `scripts/parse_results.sh`
- Template: Use the [parse_results.sh template](#) in the Templates document and match the `run_script.sh` output.
- The script should:
 1. Parse test output from the `run_script.sh`
 2. Identify Phase 1 results (broken state)
 3. Identify Phase 2 results (fixed state)
 4. Determine the overall result. Should output **"SUCCESS"** if all F2P failed and all P2P passed in phase 1, and all F2P/P2P passed in phase 2.
 5. Output results to a JSON file `test_results.json` with the required output format

Required Output Format

JSON

```
{
  "phase_1": [
    {
      "name": "path/to/test/file.f2p.test.js | test title 1",
      "status": "FAILED"
    },
    {
      "name": "path/to/test/file.p2p.test.js | test title 2",
      "status": "PASSED"
    }
  ],
  "phase_2": [
    {
      "name": "path/to/test/file.f2p.test.js | test title 1",
      "status": "PASSED"
    },
    {
      "name": "path/to/test/file.p2p.test.js | test title 2",
      "status": "PASSED"
    }
  ],
  "overall_result": "SUCCESS"
}
```

4.3 Create Base Dockerfile [\(return to TOC\)](#)

- Location: `dockerfiles/base.Dockerfile`
- The base Dockerfile is responsible for setting up the original environment as it is given to you when you open the task, and cloning the repository into `/app`. All tasks must follow the [base.Dockerfile template](#) and assume that the project root inside the container is always `/app`.

Key requirements:

- You must **not** modify the `/app` setup section or the `ENTRYPOINT` line.
- You may only change:
 - The `FROM <base-image>:<tag>` line to select an appropriate base image.

- The `<repo_url>` in the `git clone` command.
- The system dependencies section to install required packages.
- The project dependencies section to install libraries or tools needed by the tests.
- The repository used for evaluation will always be cloned into `/app` inside this base image.
- Do **not** copy scripts or patches into the image.
- Do **not** run tests inside the Dockerfile; all tests are executed at runtime using mounted `scripts/` and `patches/`.

4.4 Create Instance Dockerfile [\(return to TOC\)](#)

- Location: `dockerfiles/instance.Dockerfile`
- The instance Dockerfile extends the base image and applies the `basetoinstance.patch` to the repository located at `/app`.
- This file must follow the [instance.Dockerfile template](#) and should not copy scripts or patches into the image. All scripts and patches will be mounted at runtime.

Key requirements:

- Replace `<base_image_name>` with the name of the image built from `base.Dockerfile`. The working directory must remain `/app`, matching the base image.
- The instance Dockerfile must:
 - Reset to the latest commit inside `/app`.
 - Apply `patches/basetoinstance.patch` to produce the problem scenario.
 - As with the `base.Dockerfile`, do **not** copy scripts or patches, or run tests inside the Dockerfile; all tests are executed at runtime using mounted `scripts/` and `patches/`.

Step 5: Running the Validation Script [\(return to TOC\)](#)

To verify that all deliverables follow the project structure, that the Dockerfiles build and run successfully, that the scripts work, and that the test results are correctly generated and validated, **download the [validation script.zip](#)** and place it in your project's root directory.

Review **HOW_TO_USE.md** to run it.

The script will automatically:

1. Validate the project structure (all required files and directories)
2. Build the base and instance Docker images from your Dockerfiles
3. Run the instance container (executing `run_script.sh` and `parse_results.sh`)
4. Parse and validate the test results against the two-phase contract:
 - **Phase 1 (broken state):** F2P tests must **fail**; P2P tests must **pass**
 - **Phase 2 (fixed state):** All tests must **pass**

A human-readable summary will be printed to the terminal at the end of the run. If the run succeeds, you will see the overall SUCCESS status. If any step fails, the summary will indicate which step failed and why. Here is an example of what you will see after executing the `validation_script`:

```
2026-05-20T16:10:34 [INFO] analyzer: RESULT: SUCCESS

=====
unnamed-20260520-160812          SUCCESS
=====

Steps:
[ok] setup                        0.01s
[ok] validate_structure          0.06s
[ok] docker_build               129.13s
[ok] docker_run                 12.97s
[ok] parse_results              0.00s
[ok] validate_results           0.00s

Phase Results:
Phase 1 (broken):  F2P  0/15 pass  P2P 13/13 pass
Phase 2 (fixed):   F2P 15/15 pass  P2P 13/13 pass

Report: /mnt/c/vs/outlier/QWalpaca_issue_invention_v2/validation_script/logs/unnamed-20260520-160812/2026-05-20T21-08-12_f793b0/analysis_results.json

=====
=== Post-Run Test Coverage Check ===
Checking: /mnt/c/vs/outlier/QWalpaca_issue_invention_v2/validation_script/logs/unnamed-20260520-160812/2026-05-20T21-08-12_f793b0/test_results.json
Expected test files from test_patch.patch:
OK: 'cart.f2p.test.ts' -- 12 result(s) found
OK: 'cart.p2p.test.ts' -- 10 result(s) found
OK: 'header6.f2p.test.ts' -- 10 result(s) found
OK: 'header6.p2p.test.ts' -- 8 result(s) found
OK: 'offcanvas.f2p.test.ts' -- 4 result(s) found
OK: 'offcanvas.p2p.test.ts' -- 2 result(s) found
OK: 'shop-details.f2p.test.ts' -- 4 result(s) found
OK: 'shop-details.p2p.test.ts' -- 2 result(s) found
OK: 'swiper.p2p.test.ts' -- 4 result(s) found
All test files from test_patch.patch have results. Coverage check passed.
```

This script will generate both an `analysis_results.json` file and a terminal output. You will have to paste the table with the results in your task. The part that you will need to paste looks like this:

Shell

```
=====
<task-name> FAILED (exit XX)
=====
```

Steps:

```
[ok] setup 0.00s
[ok] validate_structure 0.00s
[ok] docker_build 0.00s
[ok] docker_run 0.00s
[ok] parse_results 0.00s
[FAIL] validate_results 0.00s
      -> <failure reason>
```

Phase Results:

```
Phase 1 (broken): F2P X/X pass P2P X/X pass
Phase 2 (fixed):  F2P X/X pass P2P X/X pass
```

Report: <path-to-analysis_results.json>

=====

=== Post-Run Test Coverage Check ===

Checking: <path-to-test_results.json>

Expected test files from test_patch.patch:

```
OK: '<test-file-1>' -- X result(s) found
OK: '<test-file-2>' -- X result(s) found
OK: '<test-file-3>' -- X result(s) found
OK: '<test-file-4>' -- X result(s) found
```

All test files from test_patch.patch have results. Coverage check passed

Step 6: Add tags [\(return to TOC\)](#)

To help categorize the task, you will add tags relevant to your task and issue. If your task requires a specific tag not listed, you may select "Other" to add it manually.

Tags *

Add the tags related to your task and issue. If your task requires a specific tag, also select "Other" to manually add them.

- ☐ JavaScript
- ☐ TypeScript
- ☒ Angular
- ☐ Node
- ☐ Redux
- ☐ Babel
- ☐ Webpack
- ☐ Jest
- ☐ localStorage
- ☐ API
- ☐ CSS
- ☐ Frontend
- ☐ Backend
- ☐ Other
- ☐ Web Frameworks
- ☐ NextJS
- ☐ React.js
- ☐ Vue.js
- ☐ JQuery
- ☐ Svelte
- ☐ Bootstrap

Step 7: Rubric Creation [\(return to TOC\)](#)

A rubric is a set of criteria that can be used to grade objectively and granularly how good a response is to a given prompt. You will create a series of criteria that exhaustively cover all the features of an ideal response to the prompt that you created. These criteria should cover all the features of a good response without being so restrictive that they exclude other possible good solutions. Therefore, you can start with your Golden Solution as a model for the rubric criteria, but make sure that you don't write them in such a way that other possible good implementations are left out.

Guidelines [\(return to TOC\)](#)

- For tips, tricks, and examples, consult the [Tips and Tricks for Creating Rubrics section in the Appendix!](#)
- You may write as many criteria as needed.
- Each criterion should match one (or more) parts of the prompt. If your criterion does not have its counterpart in the prompt, it is most likely that it is over-specific or ambiguous.
- There may be zero, one, or multiple criteria per quality dimension.
- You will assign weights to each criterion based on how important it is to the issue that you created.

- The “Correctness” quality dimension must always have at least one criterion and comprise at least 10% of your weighting.

Core Principles of Rubric Criteria [\(return to TOC\)](#)

- **Self-contained** and evaluable without external information; it should be possible to say whether the response passes or fails a criterion just by looking at the response and the criterion
- **Atomic:** Test one distinct aspect per criterion
- **Unambiguous:** Tie to measurable factors or unambiguous descriptions (avoid language like "important sections," "properly rendered," "should look good"; use enough specificity that the criterion can be graded objectively)
- **Specific:** Reference specific code fragments, UI elements, data values, file paths
- **Relevant:** Every criterion must trace back to an explicit requirement in the prompt or clearly improve the response; do not add criteria that are unrelated to the prompt or that would not make the solution better if implemented.
- Maintain **consistent granularity** across all criteria

Quality Dimensions [\(return to TOC\)](#)

Your criteria will cover the following dimensions of a good response to your prompt. All of these dimensions are optional **except for Correctness**, which must have **at least one** criterion and 10% of the overall weighting for every rubric. (You will also dedicate an additional 20% or more of the overall weight of every rubric to criteria that cover the unit tests you wrote. See [Key Requirements](#) below for more information.)

- **Correctness** – Does the generated code deliver the complete functional requirement expressed in the prompt? This dimension will be inclusive of excitability (syntactic correctness) as well as the logical correctness of the code.
 - As such, this dimension will include fail-to-pass and pass-to-pass execution criteria where applicable
- **Independence** – Is the generated code free of unnecessary components and/or dependencies that are not explicitly or implicitly required to fulfill the functional requirement expressed in the prompt?
- **Readability/Extensibility** – Is the generated code well-presented and well-documented, such that it is as easy as possible to understand and/or extend by a future collaborator?
- **Execution Efficiency** – Is the generated code free of significant inefficiencies in its utilization of memory and latency?

- **Visual Aesthetics** – Are the artifacts produced by the generated code visually appealing according to popular design standards?

Key Requirements [\(return to TOC\)](#)

- Each criterion must have an **id text (title)**, **requirement text (description)**, **weight**, and **dimension**.
- The **id text (title)** must follow the format "rubric_criteria", for example: "composer_setup".
- The sum of all weights must be 100.
- Unit testing:
 - You must add unit testing criteria (**F2P and P2P**) to your rubric under the **Correctness** dimension. The ids must always be **f2p_success** and **p2p_success**, respectively.
 - If there are multiple issues in the prompt, there must be a **f2p_success** rubric for every issue mentioned.
 - At least 20% of total weight (30-40% for non-visual or logic-heavy components) must be applied to the previously mentioned criteria (**f2p_success** + **p2p_success** = 20+ points). This is **in addition to** the 10% weighting covering the Correctness dimension.
 - Do NOT reference specific test modules or cases

Generate your rubric.json [\(return to TOC\)](#)

To use the evaluation script in the next step, you need to create a file named rubric.json. This file must follow a specific format like the one shown below:

Example:

```
JSON
[
{
  "id": "d4b5d846-c755-4aa4-a512-b98de69411b0",
  "title": "f2p_success",
  "weight": 16,
  "annotations": {
    "criteria_requirement": "All tests in tests/build.f2p.test.js pass after applying the gold patch",
    "criteria_category": "correctness"
  }
}
```

```
}  
]
```

Instead of writing the file manually, you can use the [Rubric Parser Tool](#). If you want a detailed step-by-step on how to use it you can read the guide on [how to generate the rubric.json file with the tool](#), or you can see a full tutorial in the following [video](#).

Step 8: Evaluation Script [\(return to TOC\)](#)

Before submitting, you must run your task through the evaluation suite to catch issues the validation script cannot detect. The suite checks the quality and alignment of your prompt, tests, rubrics, gold patch, Dockerfiles, and granularity.

Download the evaluation suite here: [Stargazer_Eval.zip](#)

How to run it [\(return to TOC\)](#)

- Unzip **Stargazer_Eval.zip** into your task's root directory.
Run the Stargazer Eval
- Open the **Guide/** folder and follow the HOW_TO_RUN_EVAL.md guidelines.
- Review the consolidated report. The summary table shows a **PASS/FAIL** verdict for each component: Prompt, Tests, Rubrics, Alignment, Granularity, Dockerfile, and Golden Patch.
- If any component returns **FAIL**, use the "How to Fix" table in the report to address every failed check, then re-run the master eval.
 - Note: If you modify any of the previously uploaded files (e.g., **test_patch.patch**), make sure you also re-run the validation script and upload the new files
- Only submit your task once the overall verdict is **PASS** and all components are review-ready.

This step is mandatory! A task that has not been evaluated with the Stargazer suite is considered incomplete and may be rejected during review.